

ADR-001: Neo4j + Vector Search come Graph Knowledge Engine

****Status:**** Accettato ****Data:**** 2026-06-09 ****Autore:**** Elios Scoglio ****Contesto:**** Scelta del graph database per la piattaforma AIA (Knowledge Base a Grafo, Phase 4)

Decisione

Utilizziamo Neo4j 5 Community Edition come graph database con Cypher come linguaggio di query, integrato con Qdrant per la ricerca vettoriale. Non utilizziamo Memgraph, non utilizziamo il vector index nativo di Neo4j (insufficiente per RAG production-grade).

Contesto e Problema

La piattaforma AIA necessita di un layer grafo per:

- Entity extraction — estrarre entità (persone, processi, pattern, principi) dai documenti dei clienti
- Knowledge graph traversal — navigare relazioni per arricchire le risposte RAG
- Professional knowledge — modellare principi/pattern/trade-off del consulente come entità di primo livello
- Hybrid RAG — combinare vector similarity (Qdrant) con graph traversal (Neo4j) per risposte contestuali

La scelta deve bilanciare: costo infrastruttura, complessità operativa, maturità ecosistema, performance su dataset PMI (10K-1M entità per tenant).

Alternative Valutate

1. Neo4j 5 Community Edition (SCELTA)

Cosa è: Graph database leader di mercato, 15+ anni di sviluppo, linguaggio Cypher (standard GQL ISO).

Pro:

- Cypher è il linguaggio di query a grafo più maturo e documentato al mondo
- Ecosystem vastissimo: driver ufficiali per ogni linguaggio, APOC library (450+ procedure)
- Full-text index Lucene integrato (usato per entity search)
- Community enorme: ogni problema ha già una risposta su Stack Overflow
- Docker image stabile, health check nativo, configurazione dichiarativa
- APOC supporta JSON import/export, algoritmi graph (PageRank, community detection)
- Path a Enterprise/AuraDB se serve scaling futuro
- GDS (Graph Data Science) library per analytics avanzate (community detection, similarity)

Contro:

- Community Edition: singola istanza (no cluster, no causal clustering)
- RAM-hungry: heap 1GB + page cache 512MB minimo per performance accettabili
- Cold start lento (~10-15s) per caricamento indici in memoria
- Vector search nativo (da v5.11) è basilico: no HNSW tuning, no filtering avanzato, no hybrid scoring

Costo:

Voce | Mensile

Docker container (1GB heap + 512MB pagecache) | ~0 EUR (incluso nel VPS)

Storage (~1GB per 1M nodi) | ~0 EUR (disco VPS)

Licenza Community | 0 EUR (GPLv3)

Totale | 0 EUR

2. Memgraph

Cosa e: Graph database in-memory, compatibile Cypher (subset), focus su performance real-time.

Pro:

- Velocissimo su query di traversal (in-memory puro)
- Docker image leggera (~200MB)
- Cypher-compatibile (subset)
- Stream processing integrato (Kafka, Pulsar)
- MAGE library per algoritmi graph

Contro:

- Cypher incompleto: manca supporto a diverse clausole standard (MERGE con pattern complessi, FOREACH, alcune funzioni APOC-equivalent)
- Ecosystem piccolo: documentazione limitata, community ridotta, meno risposte disponibili
- In-memory = RAM = costo: tutto il dataset deve stare in RAM. Per 500K entità + relazioni = 4-8GB RAM dedicati
- Persistenza opzionale: snapshot su disco, ma recovery lento se il container crasha
- Licenza: Community edition più restrittiva (BSL per alcune feature), Enterprise costoso
- No full-text index nativo: serve Elasticsearch/Meilisearch esterno per text search
- Meno stabile in produzione a lungo termine: meno battle-tested su workload 24/7

Costo:

Voce | Mensile

RAM dedicata (4-8GB per dataset medio) | +15-30 EUR (VPS upgrade)

Storage snapshot | ~0 EUR

Licenza Community | 0 EUR (BSL)

Elasticsearch esterno (per search) | +10 EUR

Totale | 25-40 EUR

3. Neo4j AuraDB (Cloud managed)

Cosa e: Neo4j gestito da Neo4j Inc., servizio cloud.

Pro:

- Zero ops: backup, scaling, monitoring inclusi
- Vector search integrato (AuraDB Professional)
- Causal clustering automatico

Contro:

- Costo proibitivo per PMI: AuraDB Professional parte da ~65 USD/mese per istanza minima
- Vendor lock-in cloud: non gira on-premise

- Latenza rete: ogni query attraversa Internet (vs locale su stesso VPS)
- Free tier limitatissimo: 200K nodi max, auto-pause dopo 3 giorni inattività

Costo:

Voce Mensile
AuraDB Professional (minimo) ~65 USD (~60 EUR)
Per tenant aggiuntivo (se dedicato) +65 USD ciascuno
Totale (3 tenant) ~180 EUR

4. Solo Qdrant (senza graph)

Cosa e: Usare solo il vector database per tutto, senza layer grafo.

Pro:

- Architettura più semplice (un componente in meno)
- Qdrant gestisce già vector search + metadata filtering
- Meno infrastruttura da mantenere

Contro:

- Nessun graph traversal: non puoi seguire relazioni (A → B → C) con similarità vettoriale
- Nessun reasoning chain: il modello può solo trovare "documenti simili", non "principi che si contraddicono" o "pattern che risolvono questo anti-pattern"
- Knowledge graph impossibile: le relazioni ENABLES, CONTRADICTS, MITIGATES non sono modellabili come vettori
- Professional knowledge degradato: i principi del consulente perdono struttura relazionale

Costo: 0 EUR aggiuntivi, ma valore della piattaforma significativamente ridotto.

Analisi Comparativa

Criterio	Neo4j CE	Memgraph	AuraDB	Solo Qdrant
Costo mensile	0 EUR	25-40 EUR	60-180 EUR	0 EUR
Cypher completezza	100%	~80%	100%	N/A
Ecosystem/docs	Eccellente	Buono	Eccellente	N/A
Full-text search	Nativo (Lucene)	Richiede esterno	Nativo	Metadata filter
Vector search	Basico (v5.11+)	No	Si (Professional)	Eccellente
Performance traversal	Ottimo (disco+cache)	Eccellente (in-memory)	Ottimo	N/A
Ops complexity	Bassa (Docker)	Media (RAM mgmt)	Zero	Zero
Scaling path	Enterprise/AuraDB	Enterprise	Built-in	Built-in
Graph algorithms	GDS library	MAGE library	GDS	No
Community	Enorme (15 anni)	Piccola (5 anni)	Enorme	Media
Battle-tested	LinkedIn, eBay, NASA	Pochi ref enterprise	Neo4j-backed	Qdrant-native
Multi-tenancy	Manuale (property)	Manuale (property)	Database-level	Collection-level
Cold start	10-15s	2-5s	N/A	1-2s
RAM per 500K entità	~1.5GB	~4-6GB	Managed	N/A

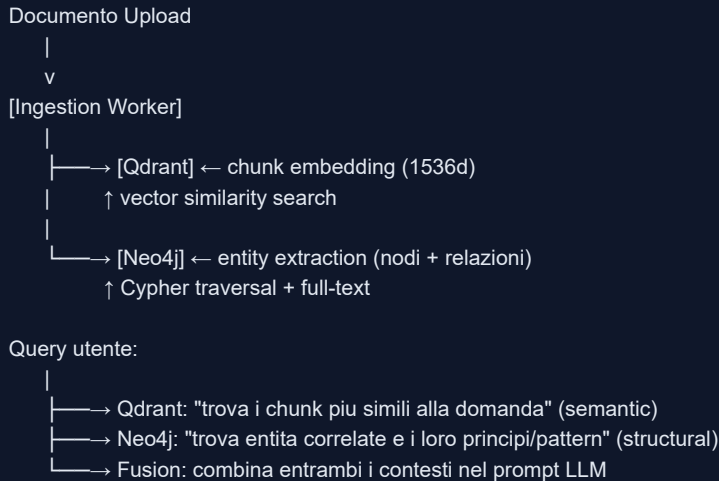
Perche l'Architettura Ibrida Neo4j + Qdrant

Il problema del "vector search nativo" di Neo4j

Neo4j 5.11+ offre vector index nativi, ma con limitazioni critiche:

- No HNSW parameter tuning: non puoi controllare ef_construction, M, o ef_search
- No hybrid scoring: non puoi combinare vector similarity + BM25 text + graph distance in una singola query
- No quantization: nessun supporto per scalar/binary quantization (riduzione costi RAM)
- Performance: ordini di grandezza più lento di Qdrant su dataset >100K vettori
- Embedding storage: occupa spazio significativo nel page cache di Neo4j (1536 float32 per nodo = 6KB)

L'architettura ibrida risolta



Vantaggi dell'ibrido:

- Qdrant fa cio che fa meglio: ricerca vettoriale ad alte prestazioni con filtering
- Neo4j fa cio che fa meglio: traversal di relazioni, reasoning chains, pattern matching
- Nessuno dei due e forzato a fare il lavoro dell'altro
- Scaling indipendente: puoi scalare Qdrant (replica set) senza toccare Neo4j

Cypher — Perché e Superiore

Confronto con alternative

Cypher vs Gremlin (TinkerPop):

```

// Cypher — dichiarativo, leggibile
MATCH (p:PRINCIPLE)-[:CONTRADICTS]->(a:ANTI_PATTERN)
WHERE p.domain = 'architecture'
RETURN p.name, a.name, a.consequence

// Gremlin — imperativo, verboso
g.V().hasLabel('PRINCIPLE').has('domain', 'architecture')
.out('CONTRADICTS').hasLabel('ANTI_PATTERN')
.project('principle', 'antipattern', 'consequence')
.by(__.inV().values('name'))
.by(__.values('name'))
.by(__.values('consequence'))
  
```

Cypher vs SPARQL (RDF):

```
// Cypher — pattern matching intuitivo
MATCH (m:METHODOLOGY)-[:SOLVES]->(p:PROCESS)
WHERE m.name = 'Strangler Fig'
RETURN p.name

// SPARQL — tripla subject-predicate-object
SELECT ?processName WHERE {
  ?m rdf:type :METHODOLOGY .
  ?m :name "Strangler Fig" .
  ?m :SOLVES ?p .
  ?p :name ?processName .
}
```

Query Cypher reali nella piattaforma

```
-- Trova tutti i principi che contraddicono un anti-pattern specifico
MATCH (ap:ANTI_PATTERN {name: "Distributed Monolith", tenantId: $tenantId})
  <-[:CONTRADICTS]-(p:PRINCIPLE)
RETURN p.name, p.description

-- Reasoning chain: da un problema al pattern che lo risolve, passando per i principi
MATCH path = (problem:CONCEPT {name: $problemName, tenantId: $tenantId})
  <-[:SOLVES]-(m:METHODOLOGY)-[:ENABLES]->(pattern:PATTERN)
RETURN path

-- Trova trade-off rilevanti per una decisione
MATCH (d:DECISION {tenantId: $tenantId})-[:INVOLVES]->(t:TRADEOFF)
WHERE d.name CONTAINS $keyword
RETURN t.name, t.left, t.right

-- Graph-enhanced RAG: dato un documento, trova principi correlati
MATCH (e:Entity {sourceDocumentId: $docId, tenantId: $tenantId})
  -[:RELATED_TO*1..2]-(related:PRINCIPLE)
RETURN DISTINCT related.name, related.description
ORDER BY related.confidence DESC
LIMIT 10
```

Configurazione Produzione

docker-compose.yml (già implementato)

```
neo4j:
  image: neo4j:5-community
  container_name: aia-neo4j
  restart: unless-stopped
  environment:
    NEO4J_AUTH: neo4j:${NEO4J_PASSWORD}
    NEO4J_PLUGINS: ["apoc"]
    NEO4J_dbms_memory_heap_max__size: 1G
    NEO4J_dbms_memory_pagecache_size: 512m
  volumes:
    - neo4j_data:/data
    - neo4j_logs:/logs
  healthcheck:
    test: ["CMD", "neo4j", "status"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Setup indici (eseguito al primo avvio)

```
-- Indice composito per tenant isolation
CREATE INDEX entity_tenant_idx IF NOT EXISTS
FOR (e:Entity) ON (e.tenantId, e.type);

-- Indice per lookup by ID
CREATE INDEX entity_id_idx IF NOT EXISTS
FOR (e:Entity) ON (e.id);

-- Indice per document lookup
CREATE INDEX entity_doc_idx IF NOT EXISTS
FOR (e:Entity) ON (e.tenantId, e.sourceDocumentId);

-- Full-text index per search
CALL db.index.fulltext.createNodeIndex(
  "entity_fulltext_idx",
  ["Entity"],
  ["name", "normalizedName"],
  {analyzer: "standard-folding"}
) IF NOT EXISTS;
```

Sizing per numero di tenant

Tenant	Entita stimate	RAM Neo4j	Storage	Note
1-5	5K-50K	1GB heap + 512MB cache	<500MB	Config attuale
5-20	50K-200K	2GB heap + 1GB cache	1-2GB	Upgrade VPS a CX41
20-50	200K-500K	4GB heap + 2GB cache	3-5GB	Valutare Enterprise
50+	500K+			Migrare a AuraDB o cluster

Costi Comparativi a 12 mesi

Scenario: 10 tenant attivi, ~100K entita totali

Soluzione	Setup	Mensile	Annuale	Note
Neo4j CE + Qdrant (attuale)		0	0 EUR	Incluso nel VPS Hetzner CX41 (50 EUR/mese)
Memgraph + Elasticsearch + Qdrant	4h config	25-40 EUR	300-480 EUR	RAM addizionale + ES
Neo4j AuraDB + Qdrant Cloud	1h config	65-130 EUR	780-1.560 EUR	Managed, zero ops

Solo Qdrant (no graph) | 0 | 0 EUR | 0 EUR | Perdita feature critica

Break-even Neo4j CE vs AuraDB: mai (CE e gratuito). Ha senso migrare ad AuraDB solo se:

- Servono 50+ tenant con SLA 99.9%
- Il costo operativo di gestire Neo4j supera il costo di AuraDB
- Servono feature Enterprise (role-based access, cluster, hot backup)

Rischi e Mitigazioni

Rischio	Impatto	Probabilità	Mitigazione
Neo4j CE non scala oltre 50 tenant	Alto	Bassa (a 12 mesi)	Monitorare, migrare a Enterprise/AuraDB quando serve
Cold start 10-15s rallenta health check	Basso	Certa	start_period: 30s nel health check, warm-up query
RAM insufficiente su VPS condiviso	Medio	Media	Monitoring heap usage, alert a 80%, upgrade VPS
Full-text index non performante su 1M+ nodi	Medio	Bassa	Fallback a Qdrant per text search, neo4j solo per traversal
Cypher injection	Alto	Bassa	Parametrized queries OVUNQUE (già implementato)

Decisione Finale — Razionale

- Costo zero — Neo4j CE gira nello stesso VPS senza costi aggiuntivi
- Cypher e il gold standard — più documentato, più maturo, standard ISO GQL
- Separazione di responsabilità — Neo4j per grafi, Qdrant per vettori, ciascuno ottimizzato
- Path di crescita chiaro — CE → Enterprise → AuraDB senza riscrittura codice
- Memgraph non aggiunge valore — il caso d'uso AIA non richiede real-time streaming né performance in-memory (le query sono batch durante ingest, non user-facing hot path)
- Vector search nativo di Neo4j è insufficiente — per RAG production serve HNSW tunabile, quantization, filtering — Qdrant fa tutto questo

Appendice — Integrazione con Professional Knowledge Graph

Il graph Neo4j non è solo un "database di entità estratte dai documenti". È anche il knowledge graph professionale del consulente: principi, pattern, anti-pattern, trade-off, metodologie.

Questo significa che quando un cliente chiede "dovremmo passare a microservizi?", il sistema:

- Qdrant → trova i chunk dei documenti del cliente rilevanti (architettura attuale, problemi)
- Neo4j → traversa il knowledge graph e trova:
 - PRINCIPLE "Bounded Context" --ENABLES--> PATTERN "Microservices"
 - ANTI_PATTERN "Distributed Monolith" --CONTRADICTS--> PATTERN "Microservices"
 - PRINCIPLE "Conway's Law" --APPLIES--> DECISION "Team Structure Before Architecture"
 - TRADEOFF "Operational Complexity vs Independent Deployability"
- Fusion → combina entrambi i contesti nel prompt LLM

Il risultato è un advisor opinionato con ragionamento strutturato, non un chatbot che rigurgita testo.

Questo è il differenziatore competitivo della piattaforma: non è "un altro RAG chatbot", è un sistema che ragiona su principi e relazioni.

